



OMNISCIA

March 26, 2026

SMART CONTRACT AUDIT REPORT

Flux Flow Finance
Uniswap V3 Core Fork



omniscia.io



info@omniscia.io



Online report: [flux-flow-finance-uniswap-v3-core-fork](https://flux-flow-finance-uniswap-v3-core-fork.omniscia.io)

Omniscia.io is one of the fastest growing and most trusted blockchain security firms and has rapidly become a true market leader. To date, our team has collectively secured over 550+ clients, detecting 2,500+ high-severity issues in widely adopted smart contracts.

Founded in France at the start of 2020, and with a track record spanning back to 2017, our team has been at the forefront of auditing smart contracts, providing expert analysis and identifying potential vulnerabilities to ensure the highest level of security of popular smart contracts, as well as complex and sophisticated decentralized protocols.

Our clients, ecosystem partners, and backers include leading ecosystem players such as L'Oréal, AvaLabs, Gnosis, Euler Finance, Kyber Network, Stake Dao, Maverick Protocol, EtherFi, Karpatkey, OlympusDao, Morpho, Fetch_ai, Arcade_xyz, Gravita, LimitBreak, and many others.

To keep up to date with all the latest news and announcements follow us on twitter @omniscia_sec.



omniscia.io



info@omniscia.io

Online report: [flux-flow-finance-uniswap-v3-core-fork](#)

Uniswap V3 Core Fork Security Audit

Audit Report Revisions

Commit Hash	Date	Audit Report Hash
59eda109cd	March 26th 2026	90d7858d88

Audit Overview

We were tasked with performing a differential audit of the Flux Flow Finance codebase and in particular their Uniswap V3 Core fork, comparing FluxFlowFi/v3-core against the canonical Uniswap/v3-core repository.

Over the course of the audit, we identified exactly one functional change: the default `feeProtocol` in `UniswapV3Pool.sol` was set to `7 + (7 << 4)`, activating a protocol fee of 1/7 (~14.3%) on both tokens from the moment of pool initialization. This value falls within the validated range `[4, 10]` enforced by the existing `setFeeProtocol` function, confirming that it operates entirely within the boundaries of the original, audited fee mechanism without introducing new control flow, access control, or mathematical logic.

The remaining deltas were all classified as non-functional and carry no security implications.

Audit Synopsis

Severity	Identified	Alleviated	Partially Alleviated	Acknowledged
Unknown	0	0	0	0
Informational	0	0	0	0
Minor	0	0	0	0
Medium	0	0	0	0
Major	0	0	0	0

During the audit, we filtered and validated a total of **0 findings utilizing static analysis** tools as well as identified a total of **0 findings during the manual review** of the codebase.

Total Findings

The list below covers each segment of the audit in depth and links to the respective chapter of the report:

- **Scope**
- **Compilation**
- **Static Analysis**
- **Manual Review**
- **Code Style**

Scope

The audit engagement encompassed a specific list of contracts that were present in the commit hash of the repository that was in scope. The tables below detail certain meta-data about the target of the security assessment and a navigation chart is present at the end that links to the relevant findings per file.

Target

- Repository: **<https://github.com/FluxFlowFi/v3-core>**
- Commit: 59eda109cde2d64308b69c4dc4bf1579005e5a57
- Language: Solidity
- Network: Fluent

Contracts Assessed

File	Total Finding(s)
contracts/libraries/BitMath.sol (BMH)	0
contracts/libraries/FullMath.sol (FMH)	0
contracts/libraries/FixedPoint96.sol (FP6)	0
contracts/libraries/FixedPoint128.sol (FP8)	0
contracts/libraries/LiquidityMath.sol (LMH)	0
contracts/libraries/LowGasSafeMath.sol (LGS)	0
contracts/NoDelegateCall.sol (NDC)	0
contracts/libraries/Oracle.sol (OEL)	0
contracts/libraries/Position.sol (PNO)	0
contracts/libraries/SafeCast.sol (SCT)	0

contracts/libraries/SwapMath.sol (SMH)	0
contracts/libraries/SqrtPriceMath.sol (SPM)	0
contracts/libraries/Tick.sol (TKC)	0
contracts/libraries/TickMath.sol (TMH)	0
contracts/libraries/TickBitmap.sol (TBP)	0
contracts/libraries/TransferHelper.sol (THR)	0
contracts/libraries/UnsafeMath.sol (UMH)	0
contracts/UniswapV3Pool.sol (UVP)	0
contracts/UniswapV3Factory.sol (UVF)	0
contracts/UniswapV3PoolDeployer.sol (UVD)	0

Compilation

The project utilizes `hardhat` as its development pipeline tool, containing an array of tests and scripts coded in TypeScript.

To compile the project, the `compile` command needs to be issued via the `npx` CLI tool to `hardhat`:

```
BASH
```

```
npx hardhat compile
```

The `hardhat` tool automatically selects Solidity version `0.7.6` based on the version specified within the `hardhat.config.ts` file.

The project does not contain any discrepancies in the Solidity versions utilized.

All `pragma` statements have been locked to `0.7.6` (`=0.7.6`), the same version utilized for our static analysis as well as optimizational review of the codebase.

During compilation with the `hardhat` pipeline, no errors were identified that relate to the syntax or bytecode size of the contracts.

Static Analysis

The execution of our static analysis toolkit identified **51 potential issues** within the codebase of which **51 were ruled out to be false positives** or negligible findings.

Manual Review

A **thorough line-by-line review** was conducted on every code delta between the FluxFlowFi/v3-core fork and the canonical Uniswap/v3-core repository to identify any deviations that could introduce malfunctions, vulnerabilities, or behavioral divergence from the audited upstream implementation.

As the project at hand implements a fork of Uniswap V3 Core, intricate care was put into ensuring that the **flow of funds within the system conforms to the specifications and restrictions** laid forth within the original Uniswap V3 protocol. We validated that **all state transitions of the system remain identical to the upstream implementation** with the sole exception of the `feeProtocol` initialization in `UniswapV3Pool.sol`, where the default was changed from `0` to `7 + (7 << 4)`. This change activates a protocol fee of 1/7 on both tokens at pool creation rather than requiring a separate `setFeeProtocol` transaction post-deployment.

We **confirmed that this value falls within the `[4, 10]` range** enforced by the existing validation logic in `setFeeProtocol`, that the fee splitting, accumulation via `protocolFees`, and collection via `collectProtocol` **remain entirely unmodified**, and that the factory owner retains the ability to adjust or disable the protocol fee at any time through the unchanged governance path.

Additionally, the system was investigated for any introduced attack surface including re-entrancy vectors, mathematical divergence, logical flaws, and access control weakening. No such issues were identified.

A total of **0 findings** were identified over the course of the manual review of which **no findings** concerned the behaviour and security of the system. The non-security related findings, such as optimizations, are included in the separate **Code Style** chapter.

Code Style

During the manual portion of the audit, we identified **no optimizations** that can be applied to the codebase that will decrease the operational cost associated with the execution of a particular function and generally ensure that the project complies with the latest best practices and standards in Solidity.

Finding Types

A description of each finding type included in the report can be found below and is linked by each respective finding. A full list of finding types Omniscia has defined will be viewable at the central audit methodology we will publish soon.

Input Sanitization

As there are no inherent guarantees to the inputs a function accepts, a set of guards should always be in place to sanitize the values passed in to a particular function.

Indeterminate Code

These types of issues arise when a linked code segment may not behave as expected, either due to mistyped code, convoluted if blocks, overlapping functions / variable names and other ambiguous statements.

Language Specific

Language specific issues arise from certain peculiarities that the Circom language boasts that discerns it from other conventional programming languages.

Curve Specific

Circom defaults to using the BN128 scalar field (a 254-bit prime field), but it also supports BSL12-381 (which has a 255-bit scalar field) and Goldilocks (with a 64-bit scalar field). However, since there are no constants denoting either the prime or the prime size in bits available in the Circom language, some Circomlib templates like `Sign` (which returns the sign of the input signal), and `AliasCheck` (used by the strict versions of `Num2Bits` and `Bits2Num`), hardcode either the BN128 prime size or some other constant related to BN128. Using these circuits with a custom prime may thus lead to unexpected results and should be avoided.

Code Style

In these types of findings, we identify whether a project conforms to a particular naming convention and whether that convention is consistent within the codebase and legible. In case of inconsistencies, we point them out under this category. Additionally, variable shadowing falls under this category as well which is identified when a local-level variable contains the same name as a toplevel variable in the circuit.

Mathematical Operations

This category is used when a mathematical issue is identified. This implies an issue with the implementation of a calculation compared to the specifications.

Logical Fault

This category is a bit broad and is meant to cover implementations that contain flaws in the way they are implemented, either due to unimplemented functionality, unaccounted-for edge cases or similar extraordinary scenarios.

Privacy Concern

This category is used when information that is meant to be kept private is made public in some way.

Proof Concern

Under-constrained signals are one of the most common issues in zero-knowledge circuits. Issues with proof generation fall under this category.

Severity Definition

In the ever-evolving world of blockchain technology, vulnerabilities continue to take on new forms and arise as more innovative projects manifest, new blockchain-level features are introduced, and novel layer-2 solutions are launched. When performing security reviews, we are tasked with classifying the various types of vulnerabilities we identify into subcategories to better aid our readers in understanding their impact.

Within this page, we will clarify what each severity level stands for and our approach in categorizing the findings we pinpoint in our audits. To note, all severity assessments are performed **as if the contract's logic cannot be upgraded** regardless of the underlying implementation.

Severity Levels

There are five distinct severity levels within our reports; `unknown`, `informational`, `minor`, `medium`, and `major`. A TL;DR overview table can be found below as well as a dedicated chapter to each severity level:

	Impact (None)	Impact (Low)	Impact (Moderate)	Impact (High)
Likelihood (None)	Informational	Informational	Informational	Informational
Likelihood (Low)	Informational	Minor	Minor	Medium
Likelihood (Moderate)	Informational	Minor	Medium	Major
Likelihood (High)	Informational	Medium	Major	Major

Unknown Severity

The `unknown` severity level is reserved for misbehaviors we observe in the codebase that cannot be quantified using the above metrics. Examples of such vulnerabilities include potentially desirable system behavior that is undocumented, reliance on external dependencies that are out-of-scope but could result in some form of vulnerability arising, use of external out-of-scope contracts that appears incorrect but cannot be pinpointed, and other such vulnerabilities.

In general, `unknown` severity level vulnerabilities require follow-up information by the project being audited and are either adjusted in severity (if valid), or marked as nullified (if invalid).

Additionally, the `unknown` severity level is sometimes assigned to centralization issues that cannot be assessed in likelihood due to their exploitation being tied to the honesty of the project's team.

Informational Severity

The `informational` severity level is dedicated to findings that do not affect the code functionally and tend to be stylistic or optimizational in nature. Certain edge cases are also set under `informational` vulnerabilities, such as overflow operations that will not manifest in the lifetime of the contract but should be guarded against as a best practice, to give an example.

Minor Severity

The **minor** severity level is meant for vulnerabilities that require functional changes in the code but tend to either have little impact or be unlikely to be recreated in a production environment. These findings can be acknowledged except for findings with a moderate impact but low likelihood which must be alleviated.

Medium Severity

The **medium** severity level is assigned to vulnerabilities that must be alleviated and have an observable impact on the overall project. These findings can only be acknowledged if the project deems them desirable behavior and we disagree with their point-of-view, instead urging them to reconsider their stance while marking the exhibit as acknowledged given that the project has ultimate say as to what vulnerabilities they end up patching in their system.

Major Severity

The **major** severity level is the maximum that can be specified for a finding and indicates a significant flaw in the code that must be alleviated.

Likelihood & Impact Assessment

As the preface chapter specifies, the blockchain space is constantly reinventing itself meaning that new vulnerabilities take place and our understanding of what security means differs year-to-year.

In order to reliably assess the likelihood and impact of a particular vulnerability, we instead apply an abstract measurement of a vulnerability's impact, duration the impact is applied for, and probability that the vulnerability would be exploited in a production environment.

Our proposed definitions are inspired by multiple sources in the security community and are as follows:

- Impact (High): A core invariant of the protocol can be broken for an extended duration.
- Impact (Moderate): A non-core invariant of the protocol can be broken for an extended duration or at scale, or an otherwise major-severity issue is reduced due to hypotheticals or external factors affecting likelihood.
- Impact (Low): A non-core invariant of the protocol can be broken with reduced likelihood or impact.
- Impact (None): A code or documentation flaw whose impact does not achieve low severity, or an issue without theoretical impact; a valuable best-practice
- Likelihood (High): A flaw in the code that can be exploited trivially and is ever-present.
- Likelihood (Moderate): A flaw in the code that requires some external factors to be exploited that are likely to manifest in practice.
- Likelihood (Low): A flaw in the code that requires multiple external factors to be exploited that may manifest in practice but would be unlikely to do so.
- Likelihood (None): A flaw in the code that requires external factors proven to be impossible in a production environment, either due to mathematical constraints, operational constraints, or system-related factors (i.e. EIP-20 tokens not being re-entrant).

Disclaimer

The following disclaimer applies to all versions of the audit report produced (preliminary / public / private) and is in effect for all past, current, and future audit reports that are produced and hosted under Omniscia:

IMPORTANT TERMS & CONDITIONS REGARDING OUR SECURITY AUDITS/REVIEWS/REPORTS AND ALL PUBLIC/PRIVATE CONTENT/DELIVERABLES

Omniscia ("Omniscia") has conducted an independent security review to verify the integrity of and highlight any vulnerabilities, bugs or errors, intentional or unintentional, that may be present in the codebase that were provided for the scope of this Engagement.

Blockchain technology and the cryptographic assets it supports are nascent technologies. This makes them extremely volatile assets. Any assessment report obtained on such volatile and nascent assets may include unpredictable results which may lead to positive or negative outcomes.

In some cases, services provided may be reliant on a variety of third parties. This security review does not constitute endorsement, agreement or acceptance for the Project and technology that was reviewed. Users relying on this security review should not consider this as having any merit for financial advice or technological due diligence in any shape, form or nature.

The veracity and accuracy of the findings presented in this report relate solely to the proficiency, competence, aptitude and discretion of our auditors. Omniscia and its employees make no guarantees, nor assurance that the contracts are free of exploits, bugs, vulnerabilities, deprecation of technologies or any system / economical / mathematical malfunction.

This audit report shall not be printed, saved, disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Omniscia.

All the information/opinions/suggestions provided in this report does not constitute financial or investment advice, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report.

Information in this report is provided 'as is'. Omniscia is under no covenant to the completeness, accuracy or solidity of the contracts reviewed. Omniscia's goal is to help reduce the attack vectors/surface and the high level of variance associated with utilizing new and consistently changing technologies.

Omniscia in no way claims any guarantee, warranty or assurance of security or functionality of the technology that was in scope for this security review.

In no event will Omniscia, its partners, employees, agents or any parties related to the design/creation of this security review be ever liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this security review.

Cryptocurrencies and all other technologies directly or indirectly related to cryptocurrencies are not standardized, highly prone to malfunction and extremely speculative by nature. No due diligence and/or safeguards may be insufficient and users should exercise maximum caution when participating and/or investing in this nascent industry.

The preparation of this security review has made all reasonable attempts to provide clear and actionable recommendations to the Project team (the "client") with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts in scope for this engagement.

It is the sole responsibility of the Project team to provide adequate levels of test and perform the necessary checks to ensure that the contracts are functioning as intended, and more specifically to ensure that the functions contained within the contracts in scope have the desired intended effects, functionalities and outcomes, as documented by the Project team.

All services, the security reports, discussions, work product, attack vectors description or any other materials, products or results of this security review engagement is provided "as is" and "as available" and with all faults, uncertainty and defects without warranty or guarantee of any kind.

Omniscia will assume no liability or responsibility for delays, errors, mistakes, or any inaccuracies of content, suggestions, materials or for any loss, delay, damage of any kind which arose as a result of this engagement/security review.

Omniscia will assume no liability or responsibility for any personal injury, property damage, of any kind whatsoever that resulted in this engagement and the customer having access to or use of the products, engineers, services, security report, or any other other materials.

For avoidance of doubt, this report, its content, access, and/or usage thereof, including any associated services or materials, shall not be considered or relied upon as any form of financial, investment, tax, legal, regulatory, or any other type of advice.